Order Management System
Analysis and Design Document : Assignment 1

Student: Asaftei Alexandru-Petru
Group: 30431

Table of Contents

Contents

1. Requirements Analysis 3
1.1 Assignment Specification 3
1.2 Functional Requirements 3
1.3 Non-functional Requirements 3
2.Use-Case Model 4
3.System Architectural Design 4
4.Class Design 5
5.Data Model 6

1. Requirements Analysis

1.1 Assignment Specification

The objective of this project is to develop a full-stack application for managing a system of persons, products, and orders. The application focuses on implementing a robust backend with Spring Boot and a dynamic frontend using Angular, ensuring data integrity through advanced validations and error handling.

1.2 Functional Requirements

The application has the following functional requirements:

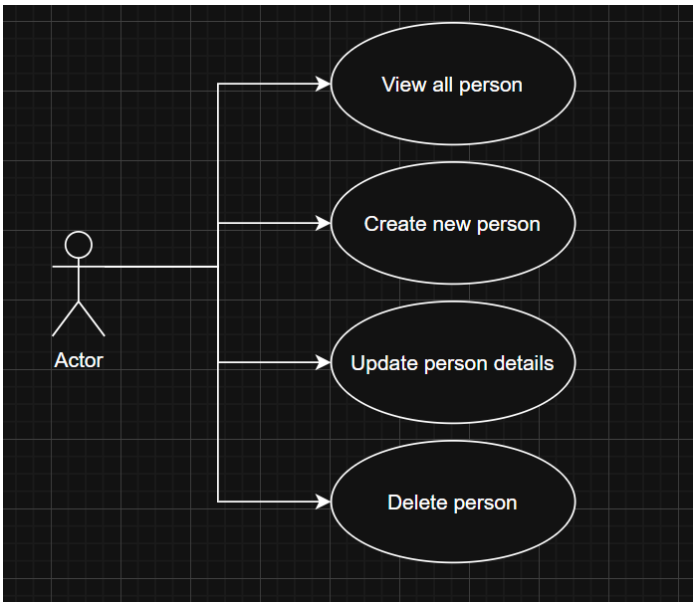
- **Person Management:** Allow users to create, view, edit, and delete entries for people in the system.
- **Input Validation:** Implement strict validation for all inputs, including name, email format, and password strength, to adhere to the specified system format.
- **Duplicate Prevention:** Ensure that business logic prevents edge cases, such as creating multiple accounts with the same email address.
- **Error Visibility:** Provide clear, user-friendly error messages in the frontend when a request fails due to validation or server errors.

1.3 Non-functional Requirements

The application has the following non-functional requirements:

- **Layered Architecture:** Use a layered architecture (Controller, Service, Repository) for better organization and separation of concerns.
- **ORM Integration:** Use Spring Data JPA (ORM) to handle all database interactions and entity mapping.
- **Dependency Injection:** Use a DI container to manage component life cycles and dependencies.
- **Global Exception Handling:** Implement a centralized exception handler to capture business logic failures and map them to appropriate HTTP responses.

2. Use-Case Model

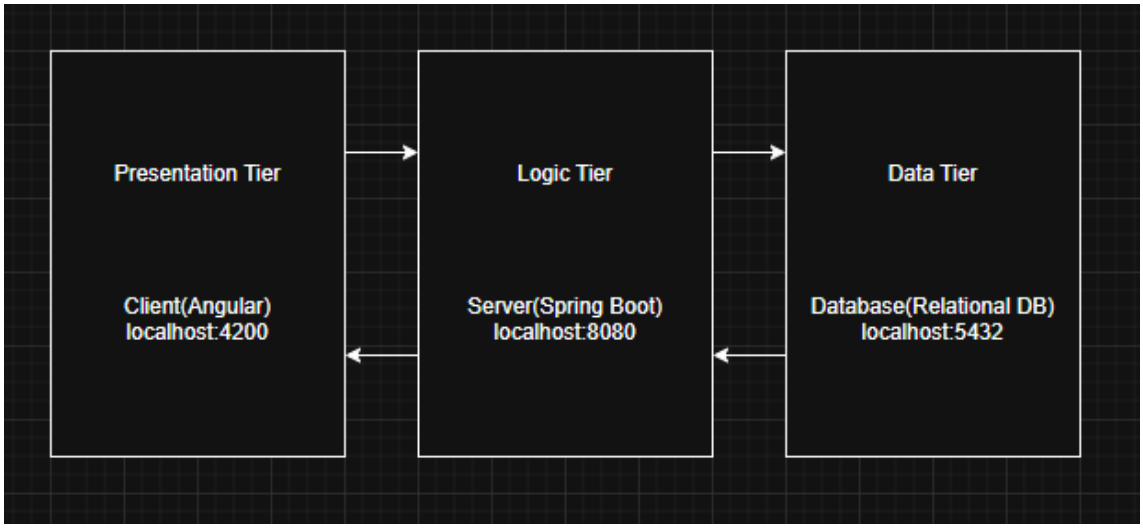


Description of the "Create Person" use-case:

- **Use case goal:** To allow the administrator to add a new person to the system after validating their credentials.
- **Primary actor:** The user of the application.
- **Main success scenario:** The user fills in the Name, Age, Email, and Password correctly. The system validates the data, saves it to the database, and the new entry appears in the list.
- **Extensions (Alternate scenario):** The database is empty. After creating the first person, the "No entries" message is replaced by the data table.
- **Failure:** The user enters an existing email or an invalid password format. The system blocks the save operation and displays a specific error message (e.g., "must be a well-formed email address").

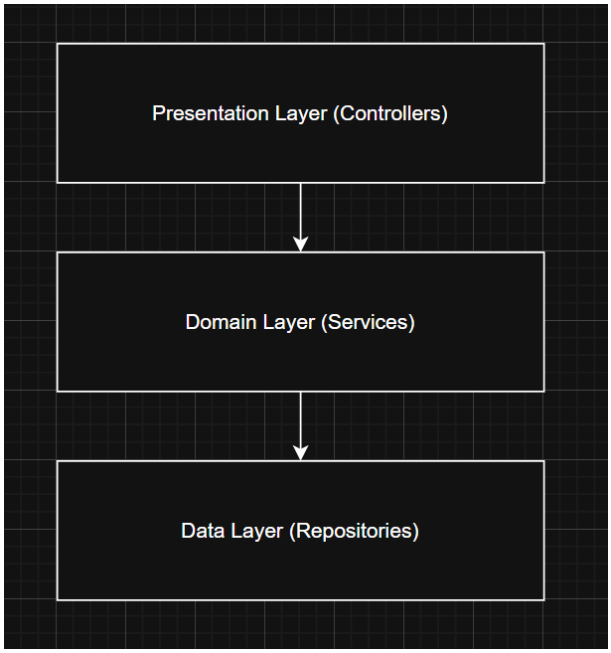
3. System Architectural Design

3.1 Architectural Pattern Description



- **Presentation Tier (Client):** Implemented as a dynamic frontend using Angular. It handles user interactions, sends requests to the backend, and displays clear, user-friendly error messages when a request fails due to validation or server errors.
- **Logic Tier (Server):** Implemented as a robust backend with Spring Boot. This tier manages the core application logic, relying on a Dependency Injection (DI) container to manage component life cycles. It enforces strict input validation and features a centralized Global Exception Handler to capture business logic failures and map them to appropriate HTTP responses.
- **Data Tier (Database):** Uses a relational database integrated with Spring Data JPA (ORM) to handle all database interactions and entity mapping.

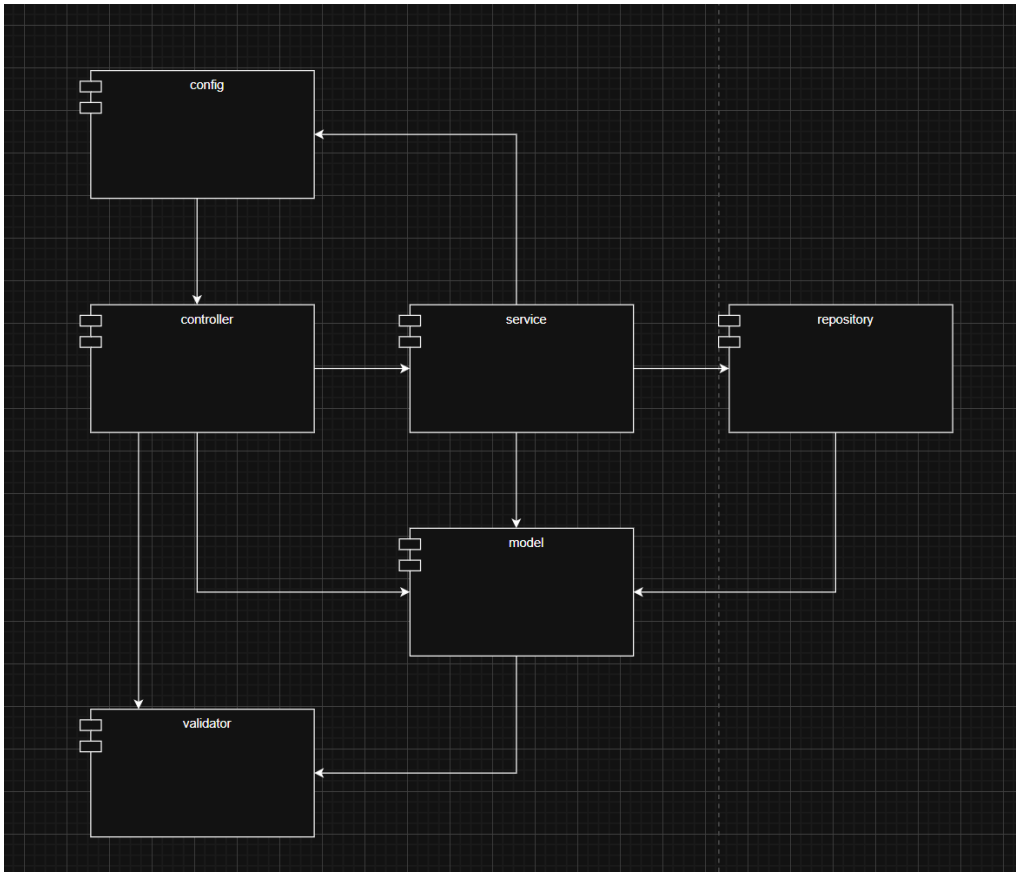
3.2 Diagrams

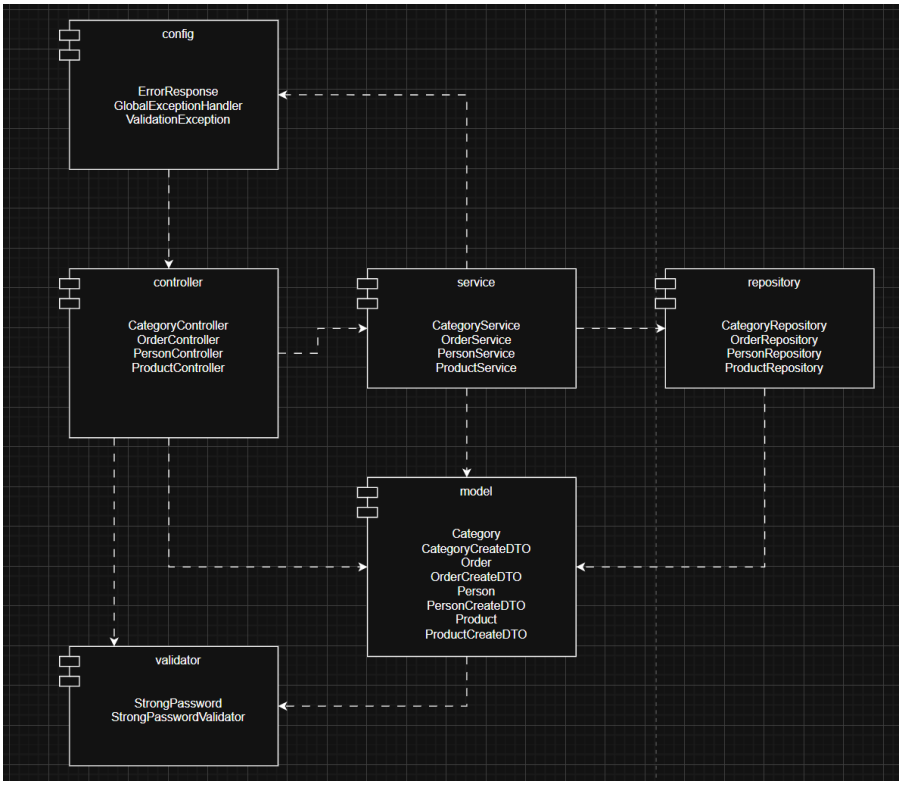


- **Presentation Layer (Controllers):** This layer acts as the entry point for REST API requests. It is responsible for handling routing, enforcing initial input validation using @Valid, and working alongside the Global Exception Handler to return consistent error responses.
- **Domain Layer (Services):** This layer holds the business logic. It ensures data integrity and prevents edge cases, such as creating multiple accounts with the same email address or assigning invalid relationships.
- **Data Layer (Repositories):** This layer provides data access. Using Spring Data JPA, it abstracts standard SQL operations into simple interfaces for seamless database manipulation and entity mapping.

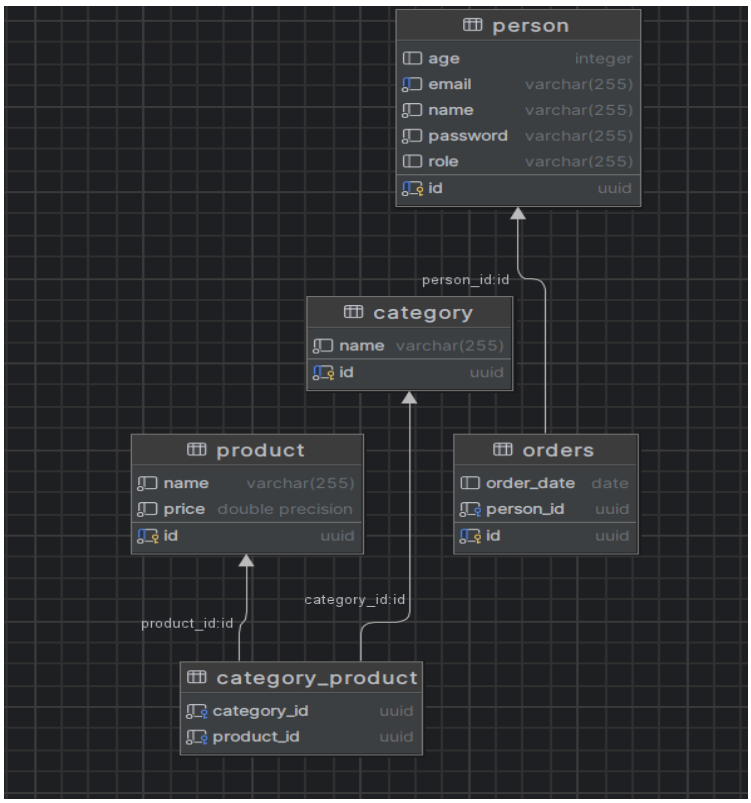
4. Class Design

4.1 Package + Class Diagram




config
ErrorResponse
GlobalExceptionHandler
ValidationException
controller
CategoryController
OrderController
PersonController
ProductController
service
CategoryService
OrderService
PersonService
ProductService
repository
CategoryRepository
OrderRepository
PersonRepository
ProductRepository
model
Category
CategoryCreateDTO
Order
OrderCreateDTO
Person
PersonCreateDTO
Product
ProductCreateDTO
validator
StrongPassword
StrongPasswordValidator

5. Data Model


person
age integer
email varchar(255)
name varchar(255)
password varchar(255)
role varchar(255)
id uuid
person_id:id
category
name varchar(255)
id uuid
product
name varchar(255)
price double precision
id uuid
orders
order_date date
person_id uuid
id uuid
category_id:id
product_id:id
category_product
category_id uuid
product_id uuid

The data model is mapped to a relational database using Spring Data JPA and Hibernate. The architecture consists of four primary entities and one junction table to resolve complex relationships:

- **Person:** The main entity storing user credentials and details (UUID, name, email, password, age, and role).
- **Orders:** Represents user purchases. There is a **One-to-Many (1:M)** relationship between person and orders. A person can place multiple orders, managed by the person_id foreign key inside the orders table.
- **Product & Category:** These tables store the application's catalog items. They share a **Many-to-Many (M:M)** relationship (a product can be in multiple categories, and a category contains multiple products).
- **Category_Product:** Because relational databases cannot handle M:M relationships directly, a junction table named category_product is used. It contains the foreign keys category_id and product_id to link the two entities together.